

Reproducibility Beyond Artifacts: Interactional Support for Collaborative Machine Learning

Zhiwei Li

Daniel J. Epstein Department of Industrial & Systems
Engineering
University of Southern California
Los Angeles, California, USA
lizhiwei@usc.edu

Carl Kesselman

Information Sciences Institute
University of Southern California
Los Angeles, California, USA
carl@isi.edu

Abstract

Machine learning (ML) reproducibility is often framed as a problem of incomplete artifact recording. This framing leads to systems that prioritize capturing datasets, code, configurations, and execution environments. However, in collaborative and interdisciplinary ML projects, reproducibility failures often arise not only from missing artifacts but from difficulties in interpreting prior work, aligning evolving components, and reconstructing experimental intent over time. Drawing on a 19-month deployment of a data-centric ML management system in a clinical research project, we identify recurring interactional breakdowns that persist despite comprehensive structural traceability. Based on these findings, we propose a two-layer socio-technical ML management system combining lifecycle-aware artifact infrastructure with an interactional layer designed to mediate coordination, explanation, and shared understanding. We discuss how an AI-mediated semantic interface reframes reproducibility as an ongoing socio-technical accomplishment rather than a static property of recorded traces, and outline implications for human-centered ML infrastructure design.

CCS Concepts

• **Human-centered computing** → **Collaborative and social computing**; • **Software and its engineering** → **Software system structures**; • **Computing methodologies** → **Machine learning**.

Keywords

Socio-technical, ML reproducibility, Interdisciplinary collaboration, Socio-technical system design, interactional support, AI-assisted development

ACM Reference Format:

Zhiwei Li and Carl Kesselman. 2026. Reproducibility Beyond Artifacts: Interactional Support for Collaborative Machine Learning. In *Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA '26)*, April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3772363.3798590>



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

CHI EA '26, Barcelona, Spain

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2281-3/26/04

<https://doi.org/10.1145/3772363.3798590>

1 Introduction

Machine learning (ML) reproducibility remains a persistent challenge, with many ML results difficult to reproduce or meaningfully reuse even when code and data are shared [6, 16]. As projects grow more complex and involve more collaborators, reproducibility problems increasingly come from how teams work rather than isolated technical bugs, particularly in high-stakes domains like healthcare, where reproducibility affects trust and clinical validity [3, 9].

Most reproducibility supporting tools focus on structural support, by which we mean the systematic recording and tracing of ML artifacts such as datasets, code, configurations, execution environments, often coupled with pipeline orchestration [5, 7]. Meanwhile, studies of collaborative ML practice emphasize the importance of communication, coordination, and shared understanding across roles, frequently advocating for improved documentation and best practices [10]. But under deadline pressure and changing goals, teams fall back on informal conversations and tacit knowledge that systems don't capture.

Together, these tensions raise a design challenge for ML infrastructure: *what human work should ML infrastructure actually support, and how?* In particular, how can systems help align the technical intent encoded in ML artifacts with the realities of collaborative practice over time?

We address this question through the deployment of DerivaML, a data-centric and lifecycle-aware ML management system [8], within a real interdisciplinary clinical research project, EyeAI [11]. By examining reproducibility in a setting where robust structural ML infrastructure was already in place, we identify recurring breakdowns centered on interactional work, such as alignment drift across evolving artifacts and the loss of context embedded in informal communication [2, 16].

Motivated by these findings, we contribute (1) an empirical characterization of interactional reproducibility breakdowns in collaborative ML, and (2) a design framework for AI-mediated interactional infrastructure. The interactional layer adds conversational support that helps teams navigate the system, understand past work, and preserve informal context. Rather than automating ML decisions, this approach scaffolds coordination, interpretation, and shared understanding, and presents a domain-agnostic design space in which ML systems proactively support reproducibility as an ongoing socio-technical accomplishment.

2 Limits of Artifact-Centric ML Management in Practice

2.1 Structural ML Management in Practice

To ground our system design exploration, we deployed Deriva-ML within an ongoing interdisciplinary clinical ML project, EyeAI. We treat this 19-month deployment as a reproducibility stress test: rather than examining ML work without tooling, we investigate challenges that persist when structural ML management infrastructure is already in place.

EyeAI reflects common collaborative ML conditions: heterogeneous clinical datasets, evolving objectives, multiple modeling strategies, and coordination across clinicians, ML engineers, and data managers. Over the 19 months, the project produced 80+ datasets, 130 executions, and 142 models, all versioned and traceable in the system.

Both authors were embedded in the project throughout, participating in regular meetings, managing data and infrastructure, and supporting experiment tracking and reproducibility. This sustained engagement provided visibility into how collaborators interpreted, reused, and struggled with prior work over time.

2.2 Recurring Breakdowns Beyond Artifacts

The structured system enabled many experiments to be rerun, compared, and reused successfully. However, during the deployment, we repeatedly encountered situations in which collaborators were unable to interpret or reuse prior experiments solely from recorded artifacts in the system. These incidents surfaced during experiment handoffs, troubleshooting sessions, configuration updating, and retrospective reviews, even when datasets, code, and execution metadata were fully traceable.

Although artifacts were present, collaborators often required additional explanation to reconstruct experimental intent or align evolving components. We documented recurring instances during meetings and troubleshooting exchanges and later revisited them, iteratively grouping them into breakdown categories through collaborative reflection between the authors. As embedded infrastructure designers and participants, our analysis reflects this situated perspective and presents analytically derived patterns rather than statistically representative findings. Table 1 summarizes the resulting analytical patterns that reveal limits of artifact-centric support.

These breakdowns cluster into three patterns. First, alignment drift: as datasets, scripts, and environments evolved separately, understanding how changes related across iterations required knowledge beyond what was structurally encoded. Second, exploratory work outside the system: under deadline pressure or during early experimentation, developers sometimes worked outside Deriva-ML, requiring later reconstruction through memory or discussion. Third, loss of shared context: as experiments accumulated, understanding their rationale increasingly depended on the original developers, making handoffs reliant on synchronous explanation.

Across all three, reproducibility failed not because artifacts were missing, but because the reasoning and context surrounding them were not readily accessible later. This suggests that sustaining reproducibility requires supporting ongoing explanation, coordination, and shared understanding, not just artifact tracking.

2.3 From Interactional Breakdowns to Design Goals

The patterns above reveal that reproducibility problems in collaborative ML rarely stem from missing artifacts. Even with comprehensive infrastructure, difficulties arise when collaborators revisit, extend, or hand off prior work, requiring explanation of decisions, relationships across iterations, and experimental intent. The challenge is not incomplete records, but a mismatch between how systems encode work and how people reason about it.

These observations suggest that reproducibility in collaborative ML work is not a static property of artifact completeness, but an ongoing socio-technical accomplishment. While structural ML management systems provide essential foundations for traceability and repeatable execution, they do not actively support the human work through which reproducibility is sustained over time. This includes sensemaking across iterations, coordination during exploration, and the articulation of informal decisions that shape experimental trajectories.

The breakdowns point to three design goals for ML management infrastructure:

Help maintain alignment as work evolves. Address recurring misalignments across configurations, dependencies, and dataset versions (Table 1, rows 1–3) by helping collaborators understand how changes relate, which dataset versions work with specific code, why parameters shifted, and how experimental goals evolved.

Support exploration within system boundaries. Reduce the need for informal or offline experimentation (rows 1 and 5) by helping people understand system expectations while experimenting, so exploratory work remains connected to structured tracking.

Preserve informal context as shared artifacts. Mitigate the loss of rationale and contextual decision-making (rows 3–5) by capturing discussions, assumptions, and negotiated trade-offs as accessible project knowledge, especially during handoffs.

Meeting these goals requires infrastructure that actively bridges the gap between human goals and system knowledge. Instead of treating coordination as something that happens outside the system, it should be supported directly alongside the artifact tracking. This shift requires designing interfaces that make encoded system knowledge conversationally accessible during ongoing work, rather than leaving interpretation to ad hoc meetings or personal memory.

3 A Two-Layer Socio-Technical ML Management System

We present a two-layer socio-technical ML management system (Figure 1) that integrates a *structural layer* for managing evolving artifacts with an *interactional layer* that mediates collaboration.¹

3.1 Structural Layer: Data-Centric, Lifecycle-Aware ML Infrastructure

The structural layer, shown on the right side of Figure 1, is implemented using Deriva-ML, which encodes ML development as a collection of formally represented artifacts that evolve over time. The system prioritizes explicit versioning, provenance, and lifecycle

¹A demonstration video showing example interaction workflows is included in the supplementary materials.

Breakdown pattern	What was missing/insufficient	Consequence in practice	Roles involved
Incomplete experiment configurations	Parameter choices, config files, and ad hoc script changes during experimentation	Couldn't reproduce runs without asking the original developer	ML engineers
Environment dependencies	Keeping data, code, and runtime aligned across framework and dependency changes	Reproducing results required manual environment reconstruction or retraining	ML engineers, data managers
Dataset version context not fully captured	Changes in inclusion criteria, preprocessing decisions, and feature engineering	Models on "similar" datasets couldn't be reproduced one-to-one	Clinicians, ML engineers, data managers
Lost rationale over time	Why experiments were run, how runs related, why results mattered	Handoffs needed meetings with original developers; hard to reuse prior work independently	ML engineers, data managers, clinicians
Provenance gaps caused by offline and exploratory work	Intermediate experiments, failed attempts, exploratory iterations done outside the system	Reuse required reverse engineering and personal memory	ML engineers

Table 1: Recurring interactional breakdowns observed when deploying Deriva-ML in a clinical project.

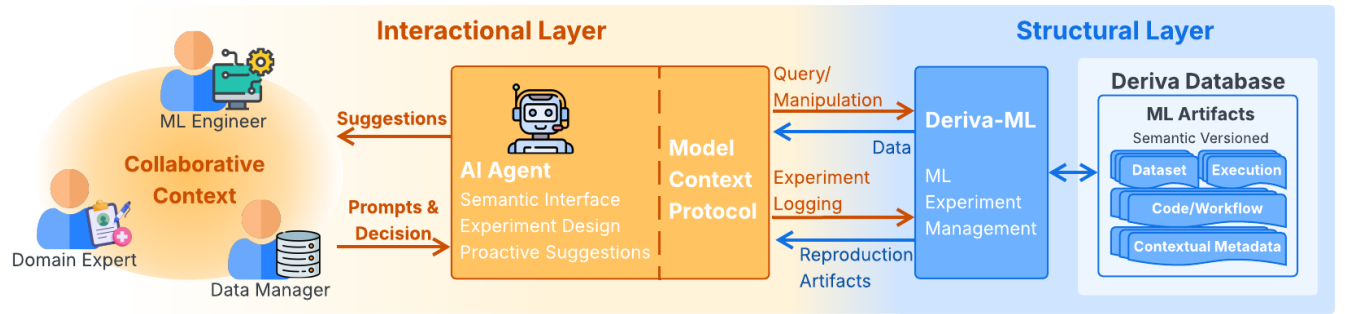


Figure 1: A two-layer socio-technical ML management system. The structural layer captures ML artifacts and their evolution. The interactional layer mediates collaboration through an AI agent that provides practice guidance, experiment suggestions, and decision support, communicating with the structural layer via the Model Context Protocol (MCP).

alignment over informal or ad hoc representations [8]. It provides a robust Python-based API, along with a rich and highly configurable browser-based user interface [4].

For clarity, Figure 1 abstracts this structure into four conceptual categories: data, code and workflows, executions, and contextual metadata, which are semantically versioned [14] and linked through explicit provenance relationships. This structure supports core reproducibility tasks such as tracing experiment lineage, comparing runs across iterations, and rerunning workflows under controlled conditions. In practice, this information is accessed through interfaces such as the Chaise UI [15], which expose artifact relationships but do not capture the informal reasoning surrounding their use.

As shown in Section 2, this form of structural traceability is essential but insufficient for sustaining reproducibility in collaborative practice. While the structural layer records what was executed and how artifacts are related, it does not support the explanation, negotiation, or sensemaking through which collaborators interpret prior work and align evolving experiments with shared intent. These limitations motivate the interactional layer described next.

3.2 Interactional Layer: AI-Mediated Support for Human-Centered ML Practice

To address the interactional gaps identified in Section 2, we introduce an interactional layer that provides context-driven scaffolding on top of the structural infrastructure. As shown on the left side of Figure 1, this layer centers on an AI agent that acts as a semantic interface between collaborators and the ML management system, helping align human goals with encoded system expectations. The agent is implemented using a large language model (Claude) with access to Deriva-ML through Model Context Protocol (MCP) tool calls: typed, schema-aware functions that expose artifact queries, provenance traversal, and execution comparisons against the live system state.

A central design principle of the interactional layer is that the agent is grounded in the semantics of the underlying ML management system. Because the agent has access to the structured representation of datasets, workflows, executions, configurations, and their relationships, it can support interaction at a level that reflects how the system itself understands ML work.

A crucial aspect of the interactional layer is that it embeds reproducibility within users' primary objective of developing effective

ML models, rather than treating it as a separate task. By supporting rapid and exploratory model development directly within the system, the interface aligns everyday practice with structured tracking. In this way, reproducibility becomes an integrated outcome of normal use, reducing the likelihood that users bypass infrastructure requirements under time pressure.

The interactional layer also supports experiment configurations as a collaborative and interpretive activity. For example, a collaborator asking which executions used a given dataset version can receive a provenance-grounded answer linking dataset history, parameter choices, and downstream runs, reconstructing the rationale that would otherwise require tracking down the original developer. Drawing on its knowledge of valid configurations, dependencies, and lifecycle constraints encoded in Deriva-ML, the agent helps practitioners set up experiment configurations, identify misalignments, and clarify how configuration choices relate to prior work.

Finally, the interactional layer treats human interactions as traceable artifacts rather than ephemeral exchanges. Interactions with the agent generate dialog transcripts that capture rationales, assumptions, and negotiated decisions that would otherwise remain ephemeral. These records accumulate as shared contexts linked to evolving artifacts that support reflection, handover, and reuse over time. In this way, the agent helps to preserve unstructured interactional knowledge as part of the ML development record, addressing the loss of shared understanding observed in long-running collaborative projects.

The interactional layer is currently implemented as a prototype and has not yet been evaluated through longitudinal deployment. It has been tested on small-scale pipeline tasks including data ingestion, dataset creation, feature generation, and ML fine-tuning configuration, but not formally validated at scale. Its role in this paper is to demonstrate how AI agents can function as interactional mediation within the ML management infrastructure, making encoded system knowledge accessible and actionable for human practice rather than merely automating technical tasks.

4 Discussion

4.1 Reproducibility as Interactional Work

Prior work on ML reproducibility has largely emphasized the recording and sharing of technical artifacts, including datasets, code, configurations, workflows, and execution environments [5, 16]. This focus underlies many ML management and MLOps tools, as well as practices such as containerization and standardized documentation, which aim to support repeatable execution and post hoc auditability [5]. Yet even when such infrastructure is in place, collaboration challenges persist: studies of ML teams document fragmented documentation, informal communication dependencies, and coordination failures across roles [1, 10, 13]. Research on MLOps further suggests that while tooling can improve certain aspects of coordination, it may also introduce new burdens related to integration, ownership, and maintenance [18].

Our findings connect these strands of work by showing how collaboration challenges surface specifically as reproducibility breakdowns, even when robust artifact management infrastructure is in place. The breakdowns we observed did not stem from missing datasets or code, but from difficulties in aligning evolving artifacts,

interpreting recorded work in light of prior decisions, and reconstructing context during reuse and handoff. From this perspective, reproducibility is not a static property of recorded traces, but an ongoing socio-technical accomplishment that depends on interactional work such as explanation, coordination, and shared understanding. This reframing suggests that ML infrastructure must support not only artifact traceability but also the human practices through which reproducibility is sustained over time.

4.2 AI Agents as Interactional Support in ML Infrastructure

Recent advances in AI-assisted development environments demonstrate the value of embedding intelligent, context-aware assistance directly into everyday work practices. AI-facilitated IDEs and emerging AI-enabled MLOps tools increasingly support tasks such as workflow configuration, debugging, and experiment management [12, 17]. These developments show that AI can operate effectively within complex technical systems without replacing human judgment.

Our work explores a different role for AI in ML infrastructure: supporting the interactional work that enables reproducibility and collaboration. While the agent assists with ML experiment configuration, its primary role is to serve as a semantic interface that helps practitioners understand system expectations, navigate complex experiment histories, and preserve informal decisions and rationales as shared context. This addresses the breakdowns from our deployment, where system knowledge existed but was difficult to access or act upon in practice.

The key is tight coupling between the conversational interface, human collaborators, and the underlying infrastructure. Because the system accesses the full project context (domain questions, artifact database, codebase), it can assist in sensemaking and decision-making. This creates a feedback loop in which conversations produce richer descriptions and rationales around artifacts, which in turn enable the system to provide better guidance when people interpret results, spot misalignments, or decide what to try next.

We argue that this feedback loop strengthens both human coordination and system traceability. Rather than treating human expertise, AI assistance, and infrastructure as separate components, the design enables them to mutually reinforce one another. In doing so, it points toward a shift from artifact-centric ML systems toward socio-technical systems that support interpretation and coordination over time.

5 Limitations and Future Work

This work is based on one interdisciplinary ML project, which limits how broadly the breakdown patterns apply. We see these patterns as diagnostic rather than project-specific, but studying teams in other domains and configurations would show how interactional challenges vary across settings. The interactional layer is an early prototype tested on small examples. Longitudinal deployment with real teams will show how conversational support affects coordination, interpretation, and handoffs in practice. Finally, our embedded role as infrastructure developers provided sustained access to collaborative ML work but may have shaped our interpretations. Studying similar systems across multiple teams will help validate and refine the proposed design space.

References

- [1] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. 2019. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 291–300.
- [2] Johannes M Bauer and Paulien M Herder. 2009. Designing socio-technical systems. In *Philosophy of technology and engineering sciences*. Elsevier, 601–630.
- [3] Andrew L Beam, Arjun K Manrai, and Marzyeh Ghassemi. 2020. Challenges to the reproducibility of machine learning models in health care. *JAMA* 323, 4 (2020), 305–306.
- [4] Alejandro Bugacov, Karl Czajkowski, Carl Kesselman, Anoop Kumar, Robert E Schuler, and Hongsuda Tangmunarunkit. 2017. Experiences with DERIVA: An asset management platform for accelerating eScience. In *2017 IEEE 13th International Conference on e-Science (e-Science)*. IEEE, 79–88.
- [5] Jose Armando Hernandez and Miguel Colom. 2025. Reproducible research policies and software/data management in scientific computing journals: a survey, discussion, and perspectives. *Frontiers in Computer Science* 6 (2025), 1491823.
- [6] Sayash Kapoor and Arvind Narayanan. 2023. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* 4, 9 (2023).
- [7] Dominik Kreuzberger, Niklas Kühl, and Sebastian Hirschl. 2023. Machine learning operations (mlops): Overview, definition, and architecture. *IEEE access* 11 (2023), 31866–31879.
- [8] Zhiwei Li, Carl Kesselman, Tran Huy Nguyen, Benjamin Yixing Xu, Kyle Bolo, and Kimberley Yu. 2025. From Data to Decision: Data-Centric Infrastructure for Reproducible ML in Collaborative eScience. In *2025 IEEE International Conference on eScience (eScience)*. 187–195. doi:10.1109/eScience65000.2025.00030
- [9] Matthew BA McDermott, Shirley Wang, Nikki Marinsek, Rajesh Ranganath, Luca Foschini, and Marzyeh Ghassemi. 2021. Reproducibility in machine learning for health research: Still a ways to go. *Science Translational Medicine* 13, 586 (2021), eabb1655.
- [10] Nadia Nahar, Shurui Zhou, Grace Lewis, and Christian Kästner. 2022. Collaboration challenges in building ml-enabled systems: Communication, documentation, engineering, and process. In *Proceedings of the 44th international conference on software engineering*. 413–425.
- [11] Van Nguyen, Sreenidhi Iyengar, Haroon Rasheed, Galo Apolo, Zhiwei Li, Aniket Kumar, Hong Nguyen, Austin Bohner, Kyle Bolo, Rahul Dhodapkar, et al. 2025. Comparison of Deep Learning and Clinician Performance for Detecting Referable Glaucoma from Fundus Photographs in a Safety Net Population. *Ophthalmology Science* 5, 4 (2025), 100751.
- [12] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirel. 2023. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590* (2023).
- [13] David Piorkowski, Soya Park, April Yi Wang, Dakuo Wang, Michael Muller, and Felix Portnoy. 2021. How ai developers overcome communication challenges in a multidisciplinary team: A case study. *Proceedings of the ACM on human-computer interaction* 5, CSCW1 (2021), 1–25.
- [14] Tom Preston-Werner. 2013. Semantic Versioning 2.0.0. <https://semver.org>. Accessed January 22, 2026.
- [15] Robert Schuler, Karl Czajkowski, Mike D'Arcy, Hongsuda Tangmunarunkit, and Carl Kesselman. 2020. Towards Co-Evolution of Data-Centric Ecosystems. In *Proceedings of the 32nd International Conference on Scientific and Statistical Database Management (Vienna, Austria) (SSDBM '20)*. Association for Computing Machinery, New York, NY, USA, Article 4, 12 pages. doi:10.1145/3400903.3400908
- [16] Harald Semmelrock, Tony Ross-Hellauer, Simone Kopeinik, Dieter Theiler, Armin Haberl, Stefan Thalmann, and Dominik Kowald. 2025. Reproducibility in machine-learning-based research: Overview, barriers, and drivers. *AI Magazine* 46, 2 (2025), e70002.
- [17] Agnia Sergeyuk, Ilya Zakharov, Ekaterina Koshchenko, and Maliheh Izadi. 2026. Human-AI experience in integrated development environments: a systematic literature review. *Empirical Software Engineering* 31, 3 (2026), 55.
- [18] Rohith Sothilingam, Vik Pant, and SK Eric. 2022. Using i* to Analyze Collaboration Challenges in MLOps Project Teams. In *ISStar*. 1–6.